SOUTENANCE D'INFORMATIQUE • ARCHITECTURE & C++

Architecture Orientée Objet d’un Solveur Thermique 1D

Conception modulaire, polymorphisme et implémentation numérique du moteur thermique_sim.exe

PRÉSENTÉ PAR

James Duburcq & Orion Routhier

EXÉCUTABLE CIBLE

`thermique_sim.exe`

Du Modèle Physique au Génie Logiciel

Éviter le code procédural monolithique.

Notre objectif principal consiste à découpler la géométrie, la physique et les algorithmes de résolution en composants C++ indépendants.

`Principe Ouvert / Fermé (SOLID)`

Modularité Stricte

Séparation claire entre maillage, matériau et solveurs.

Polymorphisme d'Exécution

Classes abstraites pures pour changer de méthode numérique.

Performance Mémoire

Conteneurs standards contigus et complexité linéaire.

Graphe des Classes et Contrats Abstraits

ENTITÉ GÉOMÉTRIQUE
mesh_1D.hpp
Discrétisation spatiale dx et nœuds.

ORCHESTRATEUR
main.cpp
Assemblage par injection.

ENTITÉ PHYSIQUE
material.hpp
Conductivité et diffusivité thermique.



INTERFACE ABSTRAITE
BoundaryCondition.hpp
↳ DirichletDC.hpp
↳ NeumannDC.hpp

INTERFACE ABSTRAITE
SpatialDiscretization.hpp
↳ DFDiscretization.hpp
↳ DFDiscretization.cpp

INTERFACE ABSTRAITE
TimeSolver.hpp
↳ IE_Solver.hpp
↳ IE_Solver.cpp

PARTIE 2 • CONCEPTION ORIENTÉE OBJET

Polymorphisme & Contrats Numériques

Isoler les conditions aux limites et les schémas de discrétisation derrière des interfaces virtuelles pures.

Hiérarchie BoundaryCondition : Dirichlet & Neumann

```cpp
// BoundaryCondition.hpp, DirichletDC.hpp & NeumannDC.hpp

class BoundaryCondition {
public:
      virtual ~BoundaryCondition() = default;
      virtual double computeBoundaryValue(
      double neighbor_temp, double dx) const = 0;
};

class DirichletDC : public BoundaryCondition {
      double T_fixed_;
public:
      double computeBoundaryValue(
      double neighbor_temp, double dx
) const override { return fixed_temperature_;
}

};
// + NeumannDC (flux thermique imposé)
```

Contrat Virtuel Pur

Le solveur manipule uniquement des pointeurs BoundaryCondition.

DirichletDC.hpp

Verrouille la température imposée sur les nœuds extrêmes.

NeumannDC.cpp

Adapte l'équation de bord selon le gradient thermique.

Destructeur Virtuel

Garantit une libération mémoire sûre lors de la destruction.

```hpp
// mesh_1D.hpp
class Mesh1D {
private:
    double length_;
    std::size_t node_count_;
    double dx_;
    std::vector<double> temperatures_;

    static double check_length(double length);
    static std::size_t check_nodes(std::size_t nodes);
```

```hpp
// material.hpp
class Material{
    private :
        double conductivite_;
        double mass_volumique_;
        double capa_ther_massique_;
        static double check_positivity(double value);

    public :
    Material(double conductivite, double mass_volumique,
double capa_ther_massique);
       ~Material() = default;
        double diffusivity() const;
```

Optimisation : Accesseurs constants (`const-correctness`) et allocation contiguë via `std::vector` pour maximiser la performance du cache CPU.

SpatialDiscretization & Différences Finies

Abstraction Pure
Contrat buildOperators assemblant M_, K_ et F_.

Assemblage Creux O(N)
3 diagonales std::vector remplies via .assign().

Opérateur Spatial Pur
Calculé sans dt depuis mat.diffusivity() et mesh.dx().

DFDiscretization.cpp

```cpp
// DFDiscretization.cpp (Assemblage Tridiagonal)
void DFDiscretization::buildOperators(const Mesh1D& mesh,
const Material& mat,
const BoundaryCondition& bc_left,
const BoundaryCondition& bc_right) {
std::size_t n = mesh.node_count();
double dx = mesh.dx();
double alpha = mat.diffusivity();
double inv_dx2 = 1.0 / (dx * dx);
// 1. En FDM, la dérivée temporelle est locale : M = I
M_.diag.assign(n, 1.0); M_.lower.assign(n - 1, 0.0); M_.upper.assign(n - 1, 0.0);
// 2. Opérateur spatial : -alpha * d²u/dx²
K_.diag.assign(n, 2.0 * alpha * inv_dx2);
K_.lower.assign(n - 1, -alpha * inv_dx2); K_.upper.assign(n - 1, -alpha * inv_dx2);
F_.assign(n, 0.0);
applyBoundaryConditions(bc_left, bc_right, dx);
}
```

Équation semi-discrétisée : $M \cdot (dU/dt) - \alpha \cdot (d^2U/dx^2) = F \rightarrow M \cdot (dU/dt) + K \cdot U = F$

Adaptation Matricielle via dynamic_cast

Identification à l'exécution

Inspection sécurisée du type réel de bc_left.

Modification locale

Ajuste uniquement les lignes 0 et n-1 des matrices.

DFDiscretization.cpp

```cpp
// DFDiscretization.cpp (applyBoundaryConditions)
if (auto dirichlet = dynamic_cast<const DirichletDC*>(&bc_left)) {
// Équation forcée : 1 * u_0 = T_fixe
M_.diag[0] = 0.0;
K_.diag[0] = 1.0; K_.upper[0] = 0.0;
F_[0] = dirichlet->computeBoundaryValue(0.0, dx);
} else if (dynamic_cast<const NeumannDC*>(&bc_left)) {
// Flux imposé : -1 * u_0 + 1 * u_1 = flux * dx
M_.diag[0] = 0.0;
K_.diag[0] = -1.0; K_.upper[0] = 1.0;
F_[0] = bc_left.computeBoundaryValue(0.0, dx);
}
```

Agrégation Faible & Euler Implicite (IE_Solver)

Agrégation par référence

TimeSolver observe SpatialDiscretization sans copie.

Précalcul setup(dt)

Assemble A = M + dt * K avant la boucle temporelle.

Algorithme de Thomas

Inversion tridiagonale en complexité linéaire O(N).

IE_Solver.cpp

```cpp
// IE_Solver.cpp (setup & step)
void IE_Solver::setup(double dt) {
const auto& M = spatial_.getMassMatrix();
const auto& K = spatial_.getStiffnessMatrix();
for (std::size_t i = 0; i < n; ++i)
A_.diag[i] = M.diag[i] + dt * K.diag[i];
}

void IE_Solver::step(Mesh1D& mesh, double dt) {
std::vector<double> Mu = multiply_matrix_vector(M, u_current);
for (std::size_t i = 0; i < n; ++i)
B[i] = Mu[i] + dt * F[i];
std::vector<double> u_next = solve_tridiagonal(A_, B);
mesh.set_temperatures(u_next);
}
```

Équation discrétisée : M · (dU/dt) + K · U = F ➔ (M + dt*K)*Unext = M*Ucurr + dt*F ➔ A*Unext = B

Pourquoi un Solveur Tridiagonal Dédié ?

$O(N)$

Complexité Temporelle et Spatiale

3 vecteurs de taille N au lieu d'une matrice N×N

vs $O(N^3)$ Pivot de Gauss

Gain d'un facteur 1 000 000 pour un maillage de N = 1000 nœuds à chaque pas de temps.

24 Ko vs 8 Mo en RAM

Les données tiennent intégralement dans le cache L1 du processeur.

Campagne de Tests Automatisée (main.cpp)

Test A : Matériaux

Comparaison Cuivre, Inox et Béton.

Test B : Accumulation

Bord droit isolé Neumann nul.

Test C & D : Flux

Refroidissement et injection 5000 W/m².

main.cpp

```
// main.cpp : Orchestration via la macro RUN_SIMULATION

Material cuivre(400.0, 8960.0, 385.0);

Mesh1D meshA1(1.0, 50, 20.0);

DirichletDC dir_100_L(BoundarySide::LEFT, 100.0);

DirichletDC dir_20_R(BoundarySide::RIGHT, 20.0);

RUN_SIMULATION("testA_cuivre.csv", meshA1, cuivre,
dir_100_L, dir_20_R,
50.0, 5000.0, 10);
```

Bilan Technique et Travail en Binôme

Robustesse C++

Zéro fuite mémoire via RAII.

Extensibilité Native

Prêt pour Crank-Nicolson ou 2D.

Synergie Binôme

Développement parallèle par contrats d'interfaces.

Test D : Injection continue de flux

Bord gauche

Flux constant $\varphi = 5000\ W \cdot m^{-2}$

Bord droit

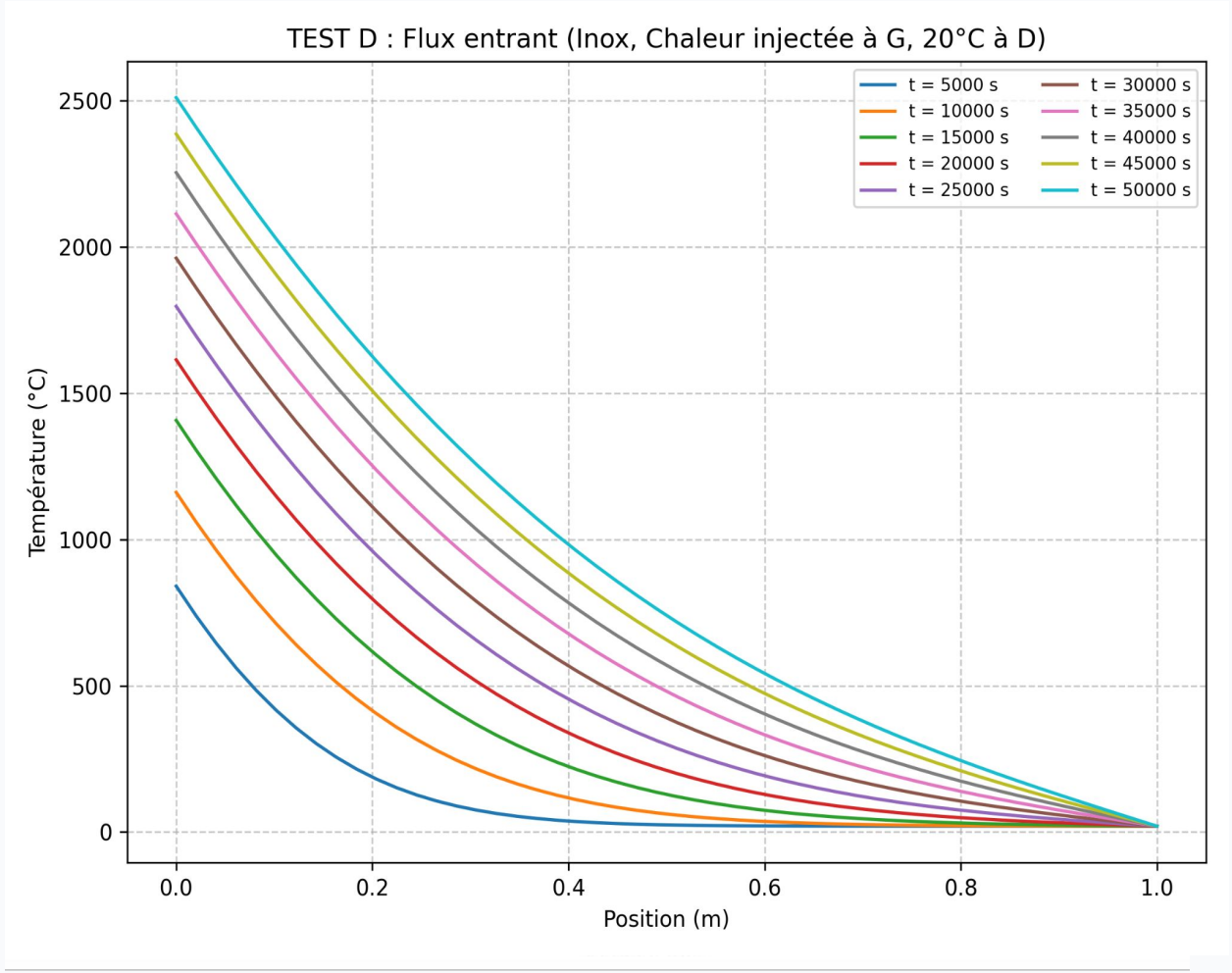
Température maintenue à **20 °C**

Inox ($k = 15\ W \cdot m^{-1} \cdot K^{-1}$)

Accumulation thermique **> 2500 °C**

Régime permanent

Profil linéaire conforme à la loi de Fourier



FIN DE LA PRÉSENTATION • THERMIQUE_SIM.EXE

Merci de votre attention

Place aux questions du jury sur l'architecture et l'implémentation C++

James Duburcq & Orion Routhier